Universidade Federal do Rio Grande do Sul
Instituto de Informática

Disciplina: INFO1120 - Desenvolvimento de Software
Professor: Marcelo Soares Pimenta

Estudo de Bad Smells e Refatoração

André Schaidhauer Luckmann 00601117
Arthur Von Groll dos Santos 00602432
Carlos Eduardo Boesel Morari 00609369
Giovani Ferreira de Alencoi 00601395

Porto Alegre
6 de outubro de 2026

1 Introdução

Um *bad smell* é um indicador de que a estrutura de um programa pode dificultar sua compreensão, manutenção e evolução. Não indica um defeito funcional por si só, fala mais sobre a estrutura do código no seu ciclo de vida. Dessa forma, bad smells podem requerer esforços elevados de análise do código e seu contexto antes de propor mudanças e novas funcionalidades, pois alguma das estruturas podem esconder certos fluxos, lógicas e antigas decisões arquiteturais.

Para resolver *bad smells*, é aplicada a ideia de refatoração visando a melhora da manutenibilidade sem alterar o comportamento observável do programa.

2 Seleção dos smells

O grupo selecionou três smells do texto-base (SMELL-1) e dois externos(SMELL-2): Feature Envy, Objetos Mutáveis, Comentários, Middle Man e Shotgun Surgery.

3 Descrição dos bad smells

3.1 Feature Envy (SMELL-1)

O bad smell *Feature Envy* trata de quando temos uma classe que tem métodos muito pouco ligados (baixa coesão) com os atributos e métodos da própria classe, e mais ligados aos métodos/atributos de outra classe. Esse bad smell pode aparecer, em um código como o exemplo abaixo:

```
1  public class Sphere {
2      private int radius, x, y;
3      private float surfaceArea;
4      private float volume;
5
6      public Sphere(int radius, int x, int y) {
7          this.radius = radius;
8          this.x = x;
9          this.y = y;
10         this.surfaceArea = this.calculateSurfaceArea();
11         this.volume = this.calculateVolume();
12     }
13
14     public int getX() {
15         return this.x;
16     }
17
18     public int getY() {
19         return this.y;
20     }
21
22     public int getRadius() {
23         return this.radius;
24     }
25
```

```
26      public float calculateSurfaceArea()
27      {
28          //...
29      }
30
31      public float calculateVolume()
32      {
33          //...
34      }
35  }
36
37  public class Drawer {
38      public void drawSphereWithShadow(GraphicsEngine engine, Sphere sphere) {
39
40          int startX = sphere.getX() - sphere.getRadius();
41          int startY = sphere.getY() - sphere.getRadius();
42          int size = sphere.getRadius() * 2;
43
44          int shadowX = startX + 5;
45          int shadowY = startY + 10;
46
47          engine.drawOval(shadowX, shadowY, size, size, GraphicsEngine.COLOR_GREY)
                ;
48          engine.drawOval(startX, startY, size, size, GraphicsEngine.COLOR_BLUE);
49      }
50  }
```

Listing 1: Feature Envy: antes da refatoração

No exemplo acima, podemos ver uma simples classe “Sphere”, que segue corretamente os padrões de código e dá a entender que se trata de uma esfera com raio, volume e área da superfície. Logo abaixo, temos uma classe “Drawer”, que aparenta ser uma classe responsável por desenhar uma dada esfera na tela para que o usuário possa visualizá-la.

De início, percebe-se que o método usado para desenhar a esfera é o “drawSphereWithShadow”, dentro de “Drawer”. No entanto, esse método não usa nenhum atributo da classe Drawer (que, inclusive, não apresenta nenhum atributo), e faz uso de vários atributos da classe “Sphere”. Isso indica, claramente, um erro de design do código: A classe “Drawer” não tem atributos e é apenas um conjunto de métodos que tratam mais sobre as classes “GraphicsEngine” e “Sphere” e poderia ser removida. Seus comportamentos poderiam ser realocados para lugares onde fariam mais sentido com comportamento muito mais coeso, o que facilitaria manutenções futuras e acarretaria em uma melhor legibilidade do código (não seria necessário acessar várias classes para entender o comportamento). Um exemplo de refatoração que causa esses impactos é o exemplo abaixo:

```
1  public interface Drawable {
2      public void draw(GraphicsEngine engine);
3  }
4
5  public class Sphere implements Drawable {
6      private int radius, x, y;
7      private float surfaceArea;
8      private float volume;
```

```
9
10      public Sphere(int radius, int x, int y) {
11          this.radius = radius;
12          this.x = x;
13          this.y = y;
14          this.surfaceArea = this.calculateSurfaceArea();
15          this.volume = this.calculateVolume();
16      }
17
18      public int getX() {
19          return this.x;
20      }
21
22      public int getY() {
23          return this.y;
24      }
25
26      public int getRadius() {
27          return this.radius;
28      }
29
30      public float calculateSurfaceArea()
31      {
32          //...
33      }
34
35      public float calculateVolume()
36      {
37          //...
38      }
39
40      @Override
41      public void draw(GraphicsEngine engine) {
42          int startX = this.getX() - this.getRadius();
43          int startY = this.getY() - this.getRadius();
44          int size = this.getRadius() * 2;
45
46          int shadowX = startX + 5;
47          int shadowY = startY + 10;
48
49          engine.drawOval(shadowX, shadowY, size, size, GraphicsEngine.COLOR_GREY)
                ;
50          engine.drawOval(startX, startY, size, size, GraphicsEngine.COLOR_BLUE);
51      }
52  }
```

Listing 2: Feature Envy: comportamento movido para Sphere

Nesse caso, foi criada uma interface “Drawable”, que, de certa forma, toma o lugar da classe “Drawer”, mas permite passar a responsabilidade para dentro da classe “Sphere”. Isso permite que “Sphere” tenha uma coesão muito maior (ela tem acesso a todas as informações da própria esfera e tudo que é necessário para desenhá-la). Isso facilita a

questão de legibilidade (tudo necessário para entender o que o método faz está na própria classe, não é necessário vagar por múltiplos arquivos diferentes) e a manutenção pode ser toda realizada ali (escopo de modificação pequeno). Além disso, essa refatoração também aplica o princípio "Tell, Don't Ask": em vez da classe externa ficar 'perguntando' pelos dados da esfera para tomar decisões, nós simplesmente "dizemos" à esfera para se desenhar, já que ela é a detentora de todas as informações necessárias para isso. Outra questão que é interessante de analizar nesse caso é que o método draw fica acoplado à **GraphicsEngine**, que poderia ser eliminado se invertessemos a dependência e fizessemos com que **GraphicsEngine** recebesse um objeto desenhável com todas as instruções de como se desenha o objeto, e a engine somente iria desenhar na tela os comandos recebidos. Porém, como a didática deste exemplo foca na resolução do smell *Feature Envy*, resolvemos manter essa implementação, mas estamos cientes das alternativas.

3.2 Objetos Mutáveis (SMELL-1)

O bad smell *Objetos Mutáveis* ocorre quando um objeto permite ser modificado após ser criado. O principal problema dessa prática é que compartilhar o objeto entre métodos gera o risco de ele ser modificado inesperadamente no fluxo do código.

Uma forma de reduzir esse risco é criar objetos imutáveis, cujo estado não muda após a construção. Em Java, a palavra reservada `final` impede a reatribuição de um objeto ou campo, mas não torna um objeto imutável por si só. Para isso, a classe não deve expor métodos que alterem seu estado; seus campos devem ser privados e finais; e referências a objetos mutáveis devem ser copiadas defensivamente ou substituídas por tipos imutáveis. Quando for necessário representar uma alteração, cria-se uma nova instância com o estado desejado. Veja o seguinte exemplo:

```java
public class Agendamento {
  private LocalDate data;

  public Agendamento(final LocalDate data) {
    this.data = data;
  }

  public void adiar(final long dias) {
    this.data = this.data.addDays(dias);
  }

  public LocalDate getData() {
    return this.data;
  }
}
```

Listing 3: Antes: objeto mutável compartilhado

Se o mesmo `Agendamento` for compartilhado por outros objetos, uma chamada a `adiar()` altera o estado observado por todos os seus clientes, o que pode ser uma alteração indesejada no fluxo do código. Abaixo, temos um exemplo que preserva o objeto original e devolve uma nova versão.

```java
public final class Agendamento {
  private final LocalDate data;

```

```
  public Agendamento(final LocalDate data) {
    this.data = data;
  }

  public Agendamento adiado(final long dias) {
    return new Agendamento(this.data.plusDays(dias));
  }

  public LocalDate getData() {
    return this.data;
  }
}
```

Listing 4: Depois: objeto imutável

Neste caso, `LocalDate` também é imutável. Se o campo fosse uma referência mutável, como `java.util.Date` ou uma lista, a classe precisaria copiar o valor no construtor e no método de acesso para não expor seu estado interno. Em resumo, ao usar objetos mutáveis, deve-se analizar se eles realmente foram projetados para serem mutáveis, e que sua mutabilidade é controlada adequadamente pelo programa. Caso exista a possibilidade de algum objetos ser alterado indevidamente, deve-se utilizar recursos para torná-lo imutável e eliminar imprevisibilidades no software.

3.3 Comentários (SMELL-1)

Comentários são um ótimo recurso para explicar o código para seres humano, porém, podem se tornar *bad smells* quando utilizados para compensar um código pouco expressivo. É interessante que o programador reflita sobre como tornar o trecho de código mais legível para que ele seja auto documentável. Dessa forma, os comentários presentes no código determinam nomenclaturas ou informações que o código por si só não explica diretamente, além de poupar a escrita de linhas de comentários que o próprio código poderia explicar.

```
x = 6 # nota da primeira prova
y = 8 # nota da segunda prova
z = 9 # nota da terceira prova
m = (x + y + z) / 3 # media das notas do semestre
```

Listing 5: Comentário que encobre código pouco expressivo

Neste trecho de código não é possível identificar o que está sendo calculado propriamente pelo código em si, por conta da falta de nomes significativos. Para suprir essa deficiência o programador inseriu comentários para explicar o fluxo, gerando um *comment smell*, dado que os comentários não se fariam necessários se os nomes escolhidos para as variáveis fossem mais expressivos. Lembrando que comentários são muito importantes para registrar alguma regra de negócio, intenção ou motivo de decisão que o código não explica por si só, porém sempre que for possível melhorar a legibilidade do código de forma que evite comentários redundantes, melhor. Já dizia o Uncle Bob que código limpo deveria soar que nem uma boa prosa escrita: seguir uma lógica estruturada que conta uma história.

```
nota_prova_1 = 6
nota_prova_2 = 8
nota_prova_3 = 9
media_semestral = (nota_prova_1 + nota_prova_2 + nota_prova_3) / 3
```

Listing 6: Nomes que tornam o comentário desnecessário

A correção de *comment smell* pode envolver múltiplas técnicas de refactoring, dado que um *comment smell* pode derivar de diversos tipos de erros de programação. No exemplo acima, aplicar nomes significativos às variáveis foi suficiente. Mas poderia requerer diferentes técnicas de Refactoring, como mudança de estruturas ou até do fluxo do código que pode ser simplificado.

3.4 Shotgun Surgery (SMELL-2)

O *Bad Smell Shotgun Surgery* acontece quando uma mudança é feita em alguma lógica e requer pequenas alterações em diferentes classes do código que usavam aquela funcionalidade. Esse problema indica que uma única responsabilidade é distribuída entre um grande número de classes. Isso pode acontecer quando muitas mudanças são feitas em uma única classe e acarreta em mudanças no seu uso pelos clientes.

4 Refatoração de Shotgun Surgery

- Usar o "Move Method" ou "Move Field" para mover comportamentos repetidos em uma classe única. Se não há classe apropriada para isso, deve-se criar uma nova.
- Se ao mover o código redundante para uma mesma classe as classes originais ficarem quase vazias, pode-se tentar refatorar as classes redundantes com o "Inline Class".
- **Vantagens**: Melhor Organização, menos código duplicado e manutenção fica mais fácil.

```
1  public class ContentService {
2    public void streamMovie(final User user) {
3      if (!user.isActive() || !"PREMIUM".equals(user.getSubscriptionTier())) {
4        throw new SecurityException("Acesso negado.");
5      }
6    }
7  }
8
9  public class DownloadController {
10   public ResponseEntity<String> downloadFile(final User user) {
11     if (!user.isActive() || !"PREMIUM".equals(user.getSubscriptionTier())) {
12       return ResponseEntity.status(403).body("Apenas para usuarios Premium.");
13     }
14     return ResponseEntity.ok("Iniciando download...");
15   }
16 }
```

Listing 7: Regra de acesso espalhada

Neste caso, se um novo plano além de PREMIUM que der acesso a esses recursos (streamMovie e downloadFile) for criado, todos os arquivos que contém essa validação inline terão que ser modificados. De certa forma, o Shotgun Surgery remete muito a código duplicado e não centralizado.

Uma proposta de refatoração que resolveria esse problema é modificar a classe de usuário para se responsabilizar por essas validações utilizadas em vários trechos de código, e depois replicar o método que valida isso nos lugares necessários:

```
public class User {
  private boolean active;
  private String subscriptionTier;

  public boolean hasPremiumAccess() {
    return this.active && ("PREMIUM".equals(this.subscriptionTier)|| "VIP".
        equals(this.subscriptionTier));
  }
}

public class ContentService {
  public void streamMovie(final User user) {
    if (!user.hasPremiumAccess()) {
      throw new SecurityException("Acesso negado.");
    }
  }
}

public class DownloadController {
  public ResponseEntity<String> downloadFile(final User user) {
    if (!user.hasPremiumAccess()) {
      return ResponseEntity.status(403).body("Apenas para usuarios Premium.");
    }
    return ResponseEntity.ok("Iniciando download...");
  }
}
```

Listing 8: Regra de acesso concentrada em User

4.1 Middle Man (SMELL-2)

O Code Smell *Middle Man* indica classes cuja única função é repassar trabalho para outra classe. Antes de remover essa camada, é necessário avaliar se ela possui uma responsabilidade que justifique sua existência. O problema pode surgir em cadeias de mensagens, por exemplo, quando clientes precisam navegar pela estrutura de objetos, ou após uma migração gradual de responsabilidades que deixa uma classe apenas delegando chamadas. Para resolver esse *smell* podemos usar o refactoring *Remove Middle Man*, que consiste nos seguintes termos:

- **Servidor**: objeto ao qual o cliente tem acesso direto.
- **Delegado**: objeto final que contém a funcionalidade solicitada.

Dois problemas aparecem com frequência:

1. A classe servidora não faz mais do que chamar o delegado e cria complexidade desnecessária.
2. Cada nova funcionalidade do delegado exige mais um método de delegação no servidor.

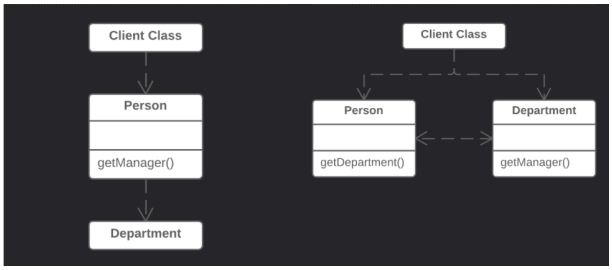


Figura 1: Refactoring do Middle Man

A refatoração consiste em fornecer acesso ao delegado e trocar chamadas que passam pelo servidor por chamadas diretas ao objeto que executa a funcionalidade. Isso reduz código repetitivo e deixa o acesso mais direto aos métodos alvo. No entanto, deve-se ponderar e argumentar esse refactoring quando a camada evita uma dependência inadequada, como nos padrões Proxy, Decorator ou Adapter. Veja a Figura 1.

4.1.1 Caso analisado no projeto da disciplina

Em determinado momento do desenvolvimento, tinhamos a classe `Player` definida da seguinte maneira: os métodos `pause`, `resume`, `stop` e `seek` apenas delegavam chamadas para `AudioEngine`.

```
1  public final class Player {
2    private final Interpreter interpreter;
3    private final AudioEngine audioEngine;
4    private final PlayerConfig playerConfig;
5
6    public Player(final Interpreter interpreter, final AudioEngine audioEngine,
         final PlayerConfig playerConfig) {
7      this.interpreter = interpreter;
8      this.audioEngine = audioEngine;
9      this.playerConfig = playerConfig;
10   }
11
12   public void play(final Music music) {
13     final String musicText = this.interpreter.interpret(music.getContent());
14     this.audioEngine.play(musicText);
15   }
16
17   public void pause() { this.audioEngine.pause(); }
18   public void resume() { this.audioEngine.resume(); }
19   public void stop() { this.audioEngine.stop(); }
20   public void seek(final int index) { this.audioEngine.seek(index); }
21 }
```

Listing 9: Player antes da análise do Middle Man

Uma alternativa seria tornar `audioEngine` público, remover os métodos delegados e fazer os clientes chamarem o mecanismo de áudio diretamente, como na Listagem 10.

```
1  public final class Player {
2    private final Interpreter interpreter;
```

```
  public final AudioEngine audioEngine;
  private final PlayerConfig playerConfig;

  public void play(final Music music) {
    final String musicText = this.interpreter.interpret(music.getContent());
    this.audioEngine.play(musicText);
  }
}

// Chamadas de Callers.
// Antes: this.player.resume();
// Depois: this.player.audioEngine.resume();
```

Listing 10: Alternativa de remoção do Middle Man

Essa alternativa elimina métodos, mas expõe uma responsabilidade interna do `Player`. O caller passa a ter liberdade para manipular a engine de áudio sem as regras aplicadas pela classe. Assim, priorizamos a integridade e a separação de responsabilidades em vez de reduzir algumas delegações.

Em um nível abaixo, `JFuguePlayer` também delega para `ManagedPlayer`, como mostra a Listagem 11.

```
public final class JFuguePlayer implements AudioEngine {
  private final ManagedPlayer managedPlayer;

  @Override
  public void pause() { this.managedPlayer.pause(); }

  @Override
  public void resume() { this.managedPlayer.resume(); }

  @Override
  public void stop() { this.managedPlayer.reset(); }

  @Override
  public void seek(final int index) {
    this.managedPlayer.seek(index);
  }
}
```

Listing 11: Delegação mantida em JFuguePlayer

Aqui também decidimos manter essa abstração porque `ManagedPlayer` pertence à biblioteca JFugue. A interface `AudioEngine` mantém a aplicação independente desse mecanismo. Se trocassemos JFugue por outra engine, como jMusic ou LeSound, os callers não precisam conhecer a nova API. Nesse contexto, a delegação atua como Adapter e evita que a remoção de um Middle Man crie um Shotgun Surgery. Porém caso tenhamos definido que todo o trabalho gira em torno de JFugue e nunca iremos poderiamos remover esse middle man e usar a biblioteca diretamente. Porém, desta vez, prezamos pela modularidade e ter um projeto agnóstico de AudioEngine.

5 Discussão

Diante das discussões do grupo, entendemos que ao escrever código, precisamos pensar sempre no contexto do código, assim das pessoas que irão ler, modificar e incrementar esse código posteriormente, de forma que as decisões de design que tomamos sejam positivas para a maior parte dos aspectos sempre que possível. Nos nossos exemplos do código do Trabalho Prático, ponderamos a eliminação do possível *Middle Man*, mas nesse caso identificamos que era importante ter uma separação agnóstica da ferramenta que usamos e também da separação de responsabilidades entre classes. Como é um projeto que dificilmente terá a AudioEngine alterada do JFugue, poderíamos lidar diretamente com ele e não com a Interface de AudioEngine sem problemas, mas como o objetivo da matéria é pensar em software de qualidade, modular e manutenível, prezamos por essas características em nossas decisões. Além disso, tivemos poucos *Bad Smells* até então, muito foi devido ao uso de Linters e ferramentas de análise estática de código como o SonarQube(versão gratuita) que nos alertam possíveis *smells* no processo de desenvolvimento. Também, devido a pipeline no github, rejeitamos alterações de código que introduzem alguns *Code Smells* na base de código, então é uma validação de qualidade a mais que nos tem ajudado a evitar problemas de *Code Smells*.

6 Conclusão

Bad smells orientam a análise de padrões de código, mas não substituem a avaliação do contexto. Como muitas coisas em desenvolvimento de software, estamos falando de Tradeoffs, é importante avaliar o motivo de cada problema ser considerado um *Bad Smell* e pensar na evolução do seu software. Assim, é importante tomar conhecimento de determinados padrões para facilitar o ciclo de desenvolvimento de software de qualidade que permita fácil compreensão e modificação.

Referências

[1] VALENTE, Marco Tulio. *Engenharia de Software Moderna*. Capítulo 9: Refactoring. Disponível em: https://engsoftmoderna.info/cap9.html. Acesso em: 4 out. 2026.

[2] FOWLER, Martin. *Refactoring: Improving the Design of Existing Code*. 2. ed. Boston: Addison-Wesley, 2018.

[3] REFACTORING.GURU. *Middle Man*. Disponível em: https://refactoring.guru/smells/middle-man. Acesso em: 4 out. 2026.

[4] REFACTORING.GURU. *Shotgun Surgery*. Disponível em: https://refactoring.guru/smells/shotgun-surgery. Acesso em: 4 out. 2026.